

LAPORAN TUGAS 2

Intelligent Multi-Agent System Prototype

CUSTOMER SERVICE AGENT

Multiagent Customer Service Hotel

Simulasi Migrasi State Mobile Agent
Menggunakan Python dan Streamlit

KELOMPOK 5

M. Al Iail Qadrillah — 26/590736/PPA/07413

Dimas Prabowo — 26/590092/PPA/07382

Monanta Alfiareza — 26/591492/PPA/07455

Frans Alwan Purba — 25/563545/PPA/07116

Dosen: Prof. Dr. Azhari, MT.

Magister Kecerdasan Artifisial - Universitas Gadjah Mada

2026

Daftar Isi

Abstrak

Status dan Cakupan Laporan

1. Identitas Kelompok

2. Problem Statement

3. State of the Art (SOTA)

4. Tujuan Proyek

5. Desain Sistem Multi Agen

6. Algoritma dan AI Integration

7. Database dan Dataset

8. Prototype dan Features

9. Evaluasi

10. Kesimpulan dan Rekomendasi

Referensi

Link Kode Program

Abstrak

Permintaan tamu hotel dapat melibatkan reservasi, kesiapan kamar, housekeeping, maintenance, dan tagihan secara bersamaan. Informasi yang tersebar pada sistem operasional menimbulkan kebutuhan koordinasi dan pembatasan kewenangan tindakan. Laporan ini menyajikan implementasi prototipe multi-agent customer service hotel dengan agen spesialis statis dan satu Mobile Investigator. Agen bergerak tersebut memindahkan state dan progres dari node logis Front Office ke Operations, melanjutkan pemeriksaan data lokal, serta mengembalikan rekomendasi kamar. Prototipe menggunakan Python, Streamlit, dua basis data SQLite lokal in-memory yang terisolasi, tombol skenario, regresi logistik lokal untuk prediksi kebutuhan keputusan manusia, dan aturan kebijakan (policy guardrails). Enam skenario mencakup gangguan AC, kebutuhan upgrade, sengketa tagihan, FAQ, housekeeping, dan rincian tagihan. Evaluasi membandingkan multi-agent statis dan varian mobile dengan kemampuan pemrosesan lokal yang setara. Seluruh implementasi telah diselesaikan dan diverifikasi melalui 70 unit test dengan tingkat kelulusan 100%. Model regresi logistik menghasilkan akurasi 91,67%, precision 100%, recall 83,33%, dan F1-score 0,9091. Pengukuran benchmark 30 iterasi berpasangan membuktikan varian mobile agent berhasil mengurangi frekuensi pesan antar-node sebesar 16,7% hingga 25,0% dengan penambahan payload serialisasi checkpoint yang sangat kecil (+27 byte), sembari menjamin kontinuitas identitas dan kepatuhan aturan bisnis secara mutlak.

Kata kunci: multi-agent system, mobile agent, customer service hotel, regresi logistik, simulasi, Streamlit.

1. Identitas Kelompok

Nama kelompok: Kelompok 5.
Judul proyek: Customer Service Agent — Multi Agent Customer Service Hotel.
Fokus tugas 2: prototipe lokal untuk memperlihatkan koordinasi agen dan simulasi migrasi state mobile agent.

No.	Nama	NIM
1	M. Al lail Qadrillah	26/590736/PPA/07413
2	Dimas Prabowo	26/590092/PPA/07382
3	Monanta Alfiaresa	26/591492/PPA/07455
4	Frans Alwan Purba	25/563545/PPA/07116

1.1 Matriks tanggung jawab

Matriks pada tugas pertama menunjukkan kontribusi bersama. Agar tanggung jawab pelaksanaan lebih jelas, Matriks peran berikut memberikan **usulan** penanggung jawab utama. Pembagian ini perlu dikonfirmasi kelompok sebelum pengumpulan; bukan catatan bahwa pekerjaan tersebut telah dilaksanakan oleh anggota yang bersangkutan. Empat anggota dapat merangkap enam peran.

Usulan pembagian peran dan keluaran.

Peran	Usulan PIC	Tanggung jawab
Leader	M. Al lail Qadrillah	Menjaga scope, keputusan desain, integrasi hasil
Researcher	Dimas Prabowo	Memeriksa literatur, produk pembanding, sitasi
Programmer	M. Al lail Qadrillah	Engine Python, migrasi, data, integrasi UI
Designer	Monanta Alfiaresa	Layout Streamlit, diagram, alur interaksi
Evaluator	Frans Alwan Purba	Skenario uji, correctness, benchmark dan analisis
Presenter	Frans Alwan Purba	Alur demonstrasi dan penjelasan hasil; anggota lain mendukung

2. Problem Statement

2.1 Masalah nyata pada operasional hotel

Customer service hotel menerima permintaan informasi, gangguan fasilitas, perubahan penempatan kamar, dan pertanyaan keuangan. Permintaan “AC rusak dan ingin pindah kamar” tidak dapat diselesaikan hanya dengan menjawab pesan. Front office harus mengetahui reservasi tamu, housekeeping harus memastikan kebersihan kamar tujuan, maintenance harus memeriksa gangguan, dan keputusan upgrade atau kompensasi memerlukan otorisasi.

Kajian Wang dkk. membahas AI dalam pariwisata melalui 177 artikel inti menurut abstrak penerbit [1]. Kajian Aransyah dkk. meninjau conversational AI dengan 71 artikel dan rentang pencarian 2010–2025 menurut abstrak yang tersedia [2]. Kedua studi mendukung relevansi domain layanan pariwisata; angka tersebut bukan bukti bahwa mobile agent hotel pasti meningkatkan kinerja.

2.2 Pain points

1. **Keterlambatan koordinasi:** front office menunggu informasi dari beberapa unit.
2. **Duplikasi pekerjaan dan biaya komunikasi:** detail kasus diteruskan ulang dan dapat menghasilkan tiket ganda.
3. **Ketidaksesuaian data:** kamar kosong pada PMS belum tentu bersih atau bebas gangguan.
4. **Batas kewenangan:** perubahan kamar rutin berbeda dari kompensasi atau sengketa tagihan.
5. **Kesulitan audit:** sulit memastikan siapa mengambil keputusan, data apa yang dipakai, dan apakah tugas selesai.
6. **Keamanan data:** agen harus mengakses data sesuai peran dan node, khususnya data reservasi dan tagihan.

2.3 Rumusan masalah dan ruang lingkup

Rumusan masalah proyek adalah: *bagaimana merancang prototipe multi-agent yang mengkoordinasikan layanan tamu lintas unit, membedakan keputusan otomatis dan manusia, serta memperlihatkan simulasi migrasi state agen yang dapat diaudit?*

Lingkungan penelitian adalah satu hotel fiktif, **Hotel Nusantara Demo**. Kanal komunikasi nyata diganti tombol skenario. Sistem berjalan lokal dalam proses python. Reservasi yang digunakan sudah tersedia; pembuatan booking baru dan perubahan tanggal tidak termasuk MVP.

Input, proses, dan output sistem.

Tahap	Rincian
Input	Tombol skenario, reservasi, occupancy, readiness, tiket, folio, dan kebijakan simulasi
Proses	Triase ML/rules, alokasi tugas, pemeriksaan evidence, migrasi bila relevan, validasi dan persetujuan

Tahap	Rincian
Output	Jawaban bersumber, tiket, rekomendasi kamar, perubahan penempatan yang sah, atau eskalasi staf
Feedback	Status tool, perubahan DB, respon tamu, progres staf, dan event audit

2.4 PEAS dan pemisahan keterlibatan manusia

PEAS untuk lingkungan customer service hotel.

Komponen	Spesifikasi
Performance	Ketepatan hasil, ketaatan policy, kontinuitas state, komunikasi, dan efisiensi komputasi
Environment	Tamu/staf simulasi, Front Office, Operations, database lokal
Actuators	Kirim pesan, buat tiket, inspeksi kandidat, ubah kamar, eskalasi
Sensors	Fixture kasus, hasil query, hasil tool, consent, status tiket, dan registry agen

Fulfillment berarti pelaksanaan fisik, seperti mengantar handuk. *Human judgment* berarti manusia mengambil alih keputusan. *Approval* adalah persetujuan atas tindakan tertentu, sedangkan *consent* adalah persetujuan tamu atas penawaran. Agen dapat berhasil mengoordinasikan fulfillment meskipun pekerjaan fisiknya tetap dilakukan staf.

3. State of the Art (SOTA)

3.1 Produk dan solusi yang relevan

SOTA pada laporan ini meliputi praktik produk dan landasan ilmiah. Fitur produk diambil dari dokumentasi pemilik; klaim pemasaran tidak diperlakukan sebagai benchmark independen atau hasil prototipe kelompok.

Perbandingan solusi dunia nyata dengan konsep proyek.

Solusi	Kemampuan terdokumentasi	Kelebihan relevan	Batas perbandingan
HiJiffy [3]	Booking assistant, virtual concierge, komunikasi multikanal dan handoff	Produk berfokus pada guest journey dan konteks hotel	Migrasi kode/state internal tidak terverifikasi; angka vendor bukan hasil proyek
Asksuite [4]	Reservation assistant, omnichannel, workflow dan peran AI	Spesialisasi layanan dan otomatisasi reservasi	Protokol koordinasi dan mobilitas internal belum diketahui dari sumber yang dibaca
Oracle OHIP [5]	Spesifikasi REST dan integrasi data operasional hotel	Kontrak enterprise sebagai acuan integrasi	Platform integrasi, bukan bukti server menerima mobile agent; akses nyata memerlukan lingkungan/credential
Mews Connector [6, 7]	API reservasi, operasi, dan tasks	Padanan konkret untuk data tamu dan tiket departemen	Endpoint mempunyai scope; bukan runtime penerima kode/state agen
Prototipe kelompok	Skenario lokal, specialist agents, checkpoint, migrasi dan audit	Alur terbuka dan dapat diinspeksi untuk pembelajaran	Cakupan sempit, data sintetis, belum divalidasi pada hotel nyata

3.2 Posisi kontribusi

Kontribusi proyek adalah keterjelasan hubungan observasi, keputusan, tindakan, dan perubahan state. Literatur mengenai Agentic AI menunjukkan bahwa sistem agen modern dapat dibangun melalui kombinasi autonomy, planning, memory, tool use, dan koordinasi antar agen. Multi-agent architecture memungkinkan beberapa agen dengan spesialisasi berbeda bekerja secara kolaboratif untuk menyelesaikan tugas yang lebih kompleks dibandingkan pendekatan single-agent [19]. Berdasarkan perspektif tersebut, proyek ini menggunakan Orchestrator sebagai pengatur alur dan beberapa specialist agent untuk menjalankan fungsi Reservation, Concierge, Billing, Operations, dan Policy. Keunggulan akademik yang dituju adalah transparansi simulasi mobilitas dan evaluasi baseline yang setara, bukan klaim mengungguli produk komersial pada skala produksi. Agent statis juga dapat meminta komputasi lokal melalui worker; karena itu pengurangan perpindahan data tidak otomatis merupakan keunggulan eksklusif mobile agent.

Landasan mobile code membedakan pilihan paradigma dan konsekuensi biaya [8]. Dalam proyek ini, mobilitas diwujudkan sebagai checkpoint aplikasi dan kelanjutan fase eksplisit dengan kode prainstal. Mobilitas tersebut berbeda dari strong mobility yang mempertahankan call stack secara transparan.

3.3 Validitas referensi tugas pertama

Identitas Wang dkk. dan jumlah 177 artikel terkonfirmasi melalui abstrak deposit penerbit; identitas Aransyah dkk. serta angka 71 didukung metadata dan abstrak indeks. Pembacaan full text kedua kajian belum dilakukan dalam riset awal. Klaim Conduit tentang kenaikan dari 90,6% menjadi 95,5% belum ditemukan sumber aslinya yang cukup jelas, sehingga tidak digunakan sebagai bukti kuantitatif.

4. Tujuan Proyek

4.1 Tujuan umum

Merancang prototipe multi-agent customer service hotel yang menampilkan spesialisasi tugas, keputusan berbasis ML dan policy, serta simulasi mobile agent dengan UI/UX yang mudah diikuti.

4.2 Tujuan khusus dan kriteria keberhasilan

1. Menunjukkan penyelesaian kasus lintas Reservation, Operations, dan Orchestrator.
2. Menjaga ID, tujuan, dan progress agen ketika state dipindahkan antar-node logis.
3. Membuktikan instance asal berhenti dan instance tujuan melanjutkan inspeksi data lokal.
4. Menggunakan regresi logistik lokal untuk memprediksi kebutuhan keputusan manusia.
5. Menegakkan policy wajib terlepas dari skor ML dan memperoleh consent sebelum perubahan kamar rutin.
6. Menyediakan dashboard status kasus, node, checkpoint, pesan, dan hasil evaluasi.
7. Membandingkan static MAS dan mobile MAS dengan data, kernel, serta policy yang sama.

Target fungsional adalah enam skenario memiliki hasil sesuai postcondition, tidak ada dua instance aktif untuk ID yang sama, tidak ada perubahan kamar tanpa consent, dan tidak ada side effect ganda. Target tersebut merupakan kriteria penerimaan, bukan hasil pengukuran saat ini. Keuntungan waktu dan komunikasi diperlakukan sebagai pertanyaan penelitian, bukan asumsi.

5. Desain Sistem Multi Agen

5.1 Spesifikasi sistem dan fitur

Python menjalankan simulator, state machine, message queue lokal, dan model ML. Streamlit menyediakan antarmuka tombol skenario dan panel monitoring. Dua node logis, FRONT_OFFICE dan OPERATIONS, mempunyai registry agen dan koneksi SQLite in-memory masing-masing. Tidak terdapat HTTP backend atau panggilan ke PMS vendor; komunikasi bisnis dilakukan sebagai pesan terstruktur di dalam simulator.

5.2 Jenis agen, klasifikasi, dan tugas

Agen	Mobilitas	Role	Tujuan dan tugas
Scenario Scout	Statis, FO	Scout	Mengamati tombol skenario dan membentuk input terstruktur; tidak memakai NLP.
Orchestrator	Statis, FO	Broker	Mengalokasikan subtask, membaca prediksi/policy, menggabungkan evidence, dan mengatur tahapan kasus.
Reservation	Statis, FO	Worker	Membaca reservasi/kandidat, membuat proposal dan menjalankan perubahan kamar yang lolos validasi.
Concierge	Statis, FO	Worker	Menjawab FAQ dari knowledge base lokal.
Billing	Statis, FO	Worker	Menampilkan folio dan menyiapkan data sengketa; tanpa kewenangan menulis nilai finansial.
Operations	Statis, OPS	Worker	Membuat/update tiket, membaca readiness, menjalankan pemeriksaan lokal pada baseline statis.
Policy	Statis, FO	Security	Mengevaluasi aturan wajib dan constraint; tool/runtime menegakkan hasil pada batas eksekusi.
Mobile Investigator	FO ke OPS	Mobile Scout/Worker	Membawa goal dan kandidat, melanjutkan inspeksi lokal setelah migrasi, lalu mengirim keputusan dan alasan.

Klasifikasi AI tidak berarti setiap agen memiliki neural network. Orchestrator dan Investigator menggunakan goal, state, reasoning berbasis constraint, dan feedback. Scenario Scout merupakan adapter input sederhana. Kecerdasan sistem terletak pada loop pengambilan keputusan dan model ML yang terukur, bukan penamaan fungsi semata.

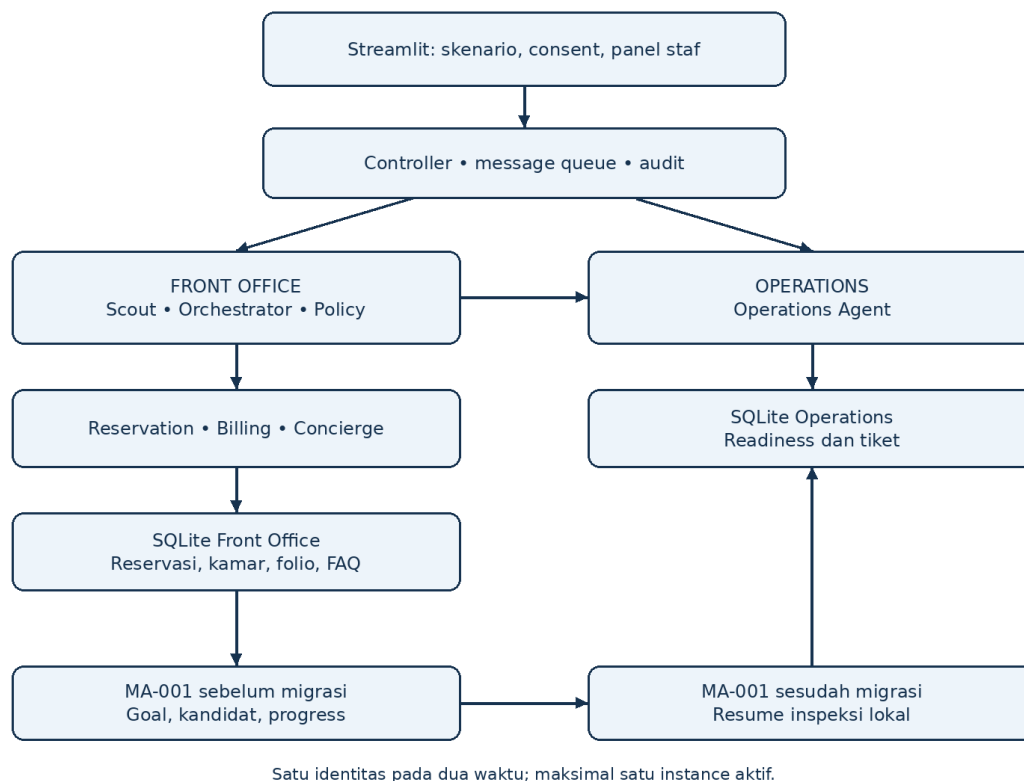
5.3 Komponen internal dan batas kewenangan

Mobile Investigator memiliki **goal** mencari kamar pengganti layak; **belief/state** berisi kandidat dan evidence yang telah diperiksa; **perception** berupa hasil query lokal; **reasoning** berupa evaluasi constraint; **planning** menentukan perpindahan ke node berkemampuan inspeksi; **action** berupa migrasi, pembacaan data, dan pengiriman hasil; **feedback** berupa hasil eksekusi yang memperbarui fase. Pembaruan state tersebut tidak otomatis berarti retraining model.

Perubahan kamar otomatis memerlukan kamar setara, bersih, kosong, tidak diblokir maintenance, tanpa kenaikan tarif, serta consent tamu. Refund, penghapusan biaya, kompensasi, upgrade di luar ketentuan, sengketa, permintaan staf, dan keadaan darurat diteruskan ke manusia. Darurat memperoleh prioritas tinggi.

5.4 Diagram arsitektur

Arsitektur simulator — dua node logis

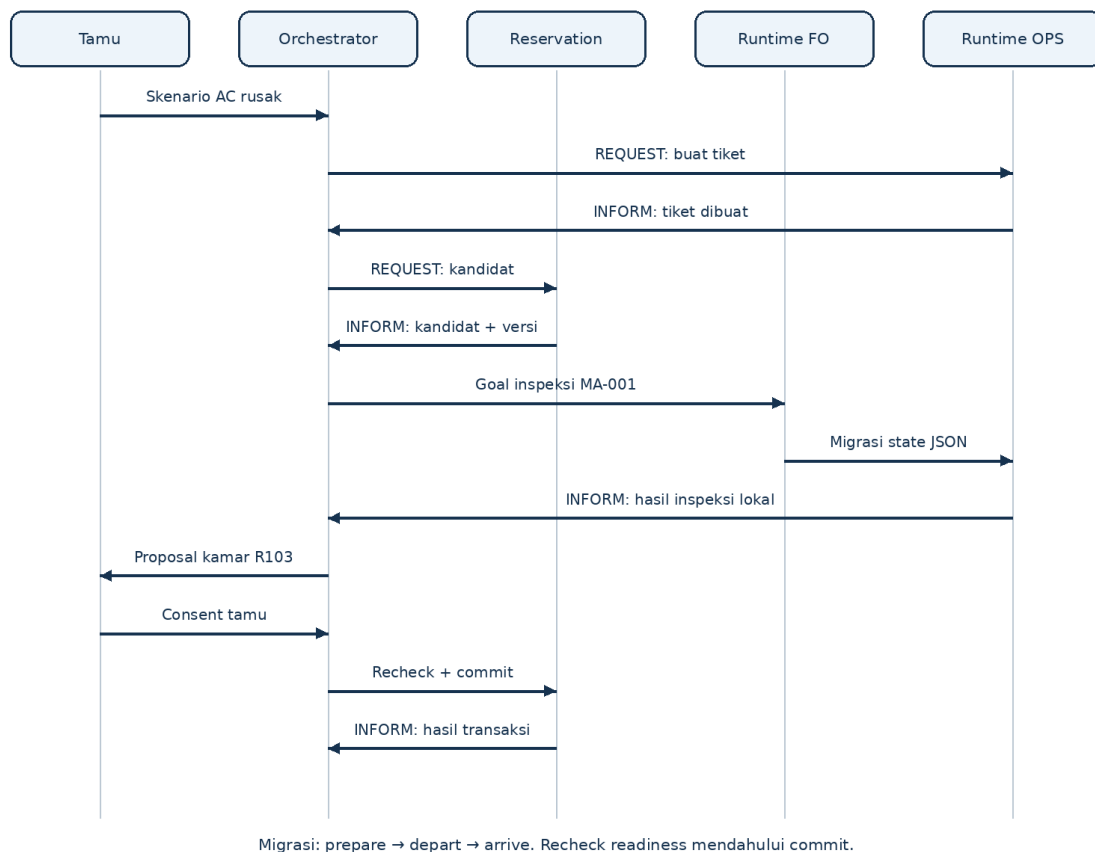


Arsitektur usulan

5.5 Koordinasi, kolaborasi, dan negosiasi terbatas

Koordinasi menggunakan supervisor–worker. Reservation mengusulkan kandidat; Investigator menilai readiness; Policy menolak kandidat yang melampaui batas; Orchestrator menyusun tindakan berikutnya. Konflik “kosong tetapi kotor” diselesaikan menggunakan sumber otoritatif per atribut, bukan voting. Pendekatan koordinasi multi-agent dapat dipandang sebagai closed-loop interaction yang melibatkan komunikasi antar agen, pengambilan keputusan, dan kontrol terhadap tindakan agen [20]. Dalam prototipe ini, Orchestrator berperan sebagai pengendali alur, sedangkan specialist agents menyediakan evidence dan proposal berdasarkan domain masing-masing. Policy Agent berfungsi sebagai constraint layer sebelum tindakan dieksekusi.

Negosiasi dalam demo bersifat proposal–accept/reject: agen menawarkan kamar yang feasible dan tamu menerima atau menolak. Bila hanya upgrade tersedia, staf memutuskan pengecualian. Sistem tidak mengimplementasikan auction, consensus terdistribusi, atau Contract Net penuh.

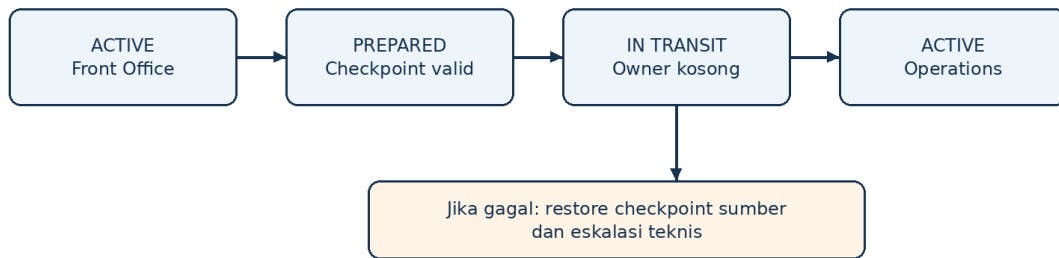
Koordinasi dan message passing — skenario S01

Message passing dan koordinasi MIMA pada kasus perpindahan kamar. Recheck readiness meminta data terbaru dari Operations sebelum commit.

5.6 Mekanisme simulasi mobile agent

Migrasi didefinisikan dalam tiga tahap:

1. **Prepare:** validasi tujuan dan capability, tetapkan fase kelanjutan, serialisasi state JSON, simpan hash serta snapshot; sumber suspended.
2. **Depart:** lepas instance dari registry aktif sumber, simpan bytes checkpoint, dan set owner menjadi kosong selama transit.
3. **Arrive:** validasi checkpoint, deserialisasi menjadi objek baru, pertahankan ID/goal/progress, daftarkan owner tujuan, lalu lanjutkan inspeksi pada step berikutnya.

Lifecycle migrasi state agen

State machine migrasi tingkat aplikasi.

Invariant utama untuk setiap identitas agen a adalah

$$N_{active}(a) \in \{0,1\}, \quad in_transit(a) \Rightarrow N_{active}(a) = 0.$$

Setelah tiba, instance tujuan harus berbeda secara objek dari instance sumber, namun ID dan progress tetap sama. Kode sudah tersedia pada kedua runtime; serialisasi JSON memindahkan data aplikasi, bukan kode atau call stack [9]. Migrasi balik tidak diperlukan; agen mengirim hasil lalu berstatus selesai di Operations.

```

{
  "schema_version": 1,
  "code_version": "investigator-v1",
  "agent_id": "MA-001",
  "case_id": "CASE-001",
  "goal": "FIND_READY_REPLACEMENT_ROOM",
  "phase": "INSPECT_LOCAL",
  "current_node": "FRONT_OFFICE",
  "candidate_room_ids": ["R102", "R103"],
  "visited_nodes": ["FRONT_OFFICE"],
  "migration_count": 0
}

```

Implementasi lengkap menyertakan snapshot kandidat, constraint, dan versi data sebagaimana PRD. Checkpoint tidak memuat koneksi SQLite, objek UI, atau model ML.

6. Algoritma dan AI Integration

6.1 Pemetaan metode dan status penggunaan

Cakupan ML, DL, RL, rule-based, dan GNN.

Metode	Peran	Status dalam proyek
ML	Prediksi kebutuhan keputusan manusia menggunakan regresi logistik	Terlatih dan terintegrasi penuh; pipeline StandardScaler + LogisticRegression dilatih lokal secara deterministik tanpa koneksi eksternal.
Rule-based	Policy wajib, readiness, izin perubahan, dan local access	Dispesifikasikan sebagai komponen inti MVP
DL	Pemahaman chat/representasi teks jika input bebas ditambahkan	Rancangan pengembangan; tidak diperlukan tombol skenario
RL	Pemilihan stay/migrate berdasarkan biaya/telemetry	Rancangan pengembangan; MVP memilih node berdasarkan capability
GNN	Prediksi biaya/ranking runtime pada graph banyak node	Rancangan pengembangan; dua node belum memerlukan GNN

6.2 Regresi logistik untuk human judgment

Pada tugas pertama, fitur yang dicontohkan adalah kompleksitas intent x_1 dan risiko x_2 . Model menghitung

$$z = b + w_1 x_1 + w_2 x_2, P(H = 1 | x) = \sigma(z) = \frac{1}{1 + \exp(-z)}.$$

Dengan bobot ilustratif $w_1 = 0,8$, $w_2 = 2$, $b = -2$, serta $x_1 = 4$ dan $x_2 = 1$, diperoleh $z = 3,2$ dan $P(H = 1 | x) \approx 0,9608$. Pada threshold $\tau = 0,80$, keputusan model adalah eskalasi. Perhitungan ini benar, tetapi tidak membuktikan model sudah dilatih atau akurasinya 96,08%.

Untuk MVP, fitur diperluas menjadi

$$x = (\text{complexity}, \text{risk}, \text{failed attempts}, \text{context missing}).$$

Model dirancang sebagai StandardScaler diikuti LogisticRegression dengan $C = 1$, maksimum 1.000 iterasi, dan random_state=42. Scaler dan classifier di-fit hanya pada data train (36 sampel train, 12 sampel test stratified). Parameter hasil training aktual yang diperoleh sistem adalah intercept $b = -0,0876$, dan bobot koefisien: $w_{\text{complexity}} = 1,4557$, $w_{\text{risk}} = 1,4098$, $w_{\text{failed attempts}} = 0,9490$, dan $w_{\text{context missing}} = 1,3720$. Parameter normalisasi scaler adalah mean = [2,3611, 0,5000, 1,0278, 0,5000] dan scale = [1,0840, 0,5000, 0,8329, 0,5000]. Dataset training memiliki hash SHA-256: baa22175abab5bbfbf0aaf9c7b4147045f7c5de2d60eb0571d8cae3c80a6535d. Seluruh parameter ini tersimpan secara deterministik dan diekspor sebagai artefak model pada sistem.

6.3 Integrasi policy dan prediksi

Keputusan akhir didefinisikan sebagai

$$D(x, c) = \{Human, jika\ policy\ wajib\ aktif\ pada\ konteks\ c, Human, jika\ P(H = 1 | x) \geq \tau, Agent\ dalam\ batas\ izin, selainnya.\}$$

Threshold 0,80 digunakan sebagai parameter demonstrasi, bukan threshold optimum hasil eksperimen. Keputusan statistik dan tindakan kebijakan merupakan dua lapisan berbeda [11]. Sistem menampilkan probabilitas, threshold, alasan policy, dan sumber keputusan secara terpisah.

Untuk setiap kandidat kamar r , eligibility rutin dirumuskan sebagai

$$\begin{aligned} ready(r) &= clean(r) \wedge \neg blocked(r), eligible(r) \\ &= vacant(r) \wedge ready(r) \wedge sameType(r) \wedge noExtraFee(r). \end{aligned}$$

Eksekusi memerlukan eligibility, consent yang sesuai proposal, versi data yang masih berlaku, dan tidak adanya human-review flag. Candidate dengan tipe/tarif lebih tinggi hanya menjadi alternatif untuk keputusan staf.

6.4 Algoritme proses utama

```
request = load_selected_scenario()
p_human = trained_local_model.predict_proba(request.features)
decision = policy(request.flags, p_human, threshold=0.80)
ticket = operations.create_ticket(idempotency_key=case_id)
candidates = reservation.find_candidates()

if mode == "mobile":
    agent = create_investigator(candidates, goal)
    prepare_checkpoint(agent, destination="OPERATIONS")
    depart_source(agent)
    resumed_agent = arrive_and_reconstruct()
    evidence = resumed_agent.inspect_local_candidates()
else:
    evidence = operations.inspect_batch(candidates, goal)

if decision.requires_human or evidence.only_upgrade:
    escalate_with_evidence()
else:
    proposal = choose_first_eligible_room(evidence)
    wait_for_guest_consent(proposal)
    recheck_and_commit_or_escalate(proposal)
```

Pseudocode ini ringkasan logis. Pada simulator, setiap langkah antrean atau tahap migrasi dijalankan terpisah melalui tombol agar progress dapat diamati.

6.5 DL sebagai pengembangan input bahasa alami

Jika chat bebas diperlukan di masa depan, encoder transformer pralatih dapat memetakan teks m menjadi representasi $h = f_{\theta}(m)$, kemudian classifier memprediksi intent atau retrieval memilih FAQ. Model multilingual MiniLM adalah salah satu kandidat sumber terbuka [14]. Penggunaan encoder

pralatih bukan training DL dari nol. Pada MVP, input berupa skenario terstruktur sehingga DL tidak dijalankan dan tidak ada unduhan model.

6.6 RL sebagai pengembangan pemilihan lokasi

Pemilihan lokasi dapat dikembangkan sebagai MDP dengan state berisi lokasi, fase, langkah tersisa, beban node, latency, dan ukuran checkpoint; aksi berupa eksekusi lokal, RPC, atau migrasi ke node berizin. Reward konseptual:

$$R = \lambda_s I_{sukses} - \alpha T - \beta B - \gamma F,$$

dengan T waktu simulasi, B volume komunikasi, dan F kegagalan. Koefisien harus dinormalisasi dan diuji. Pembaruan Q-learning dapat ditulis

$$Q(s, a) \leftarrow Q(s, a) + \eta \left[r + \gamma_q \max_a Q(s', a') - Q(s, a) \right].$$

Eksperimen RL nantinya memerlukan seed train/test dan baseline heuristic yang berbeda, serta action mask untuk node terlarang. Framework simulator seperti Gymnasium dapat membantu mendefinisikan observation/action space [15]. MVP belum mengimplementasikan RL.

6.7 GNN sebagai pengembangan topologi

Pada banyak runtime, graph $G = (V, E)$ dapat memodelkan node dengan fitur load dan capability, serta edge dengan latency dan bandwidth. GNN dapat memprediksi biaya atau ranking node. Salah satu bentuk pembaruan GCN adalah

$$H^{(l+1)} = \sigma(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} W^{(l)})$$

[16]. Diperlukan data training, loss, dan evaluasi topologi held-out. Menggambar node dan panah tidak berarti menggunakan GNN. Karena MVP hanya dua node, graph neural model belum menjadi komponen implementasi.

7. Database dan Dataset

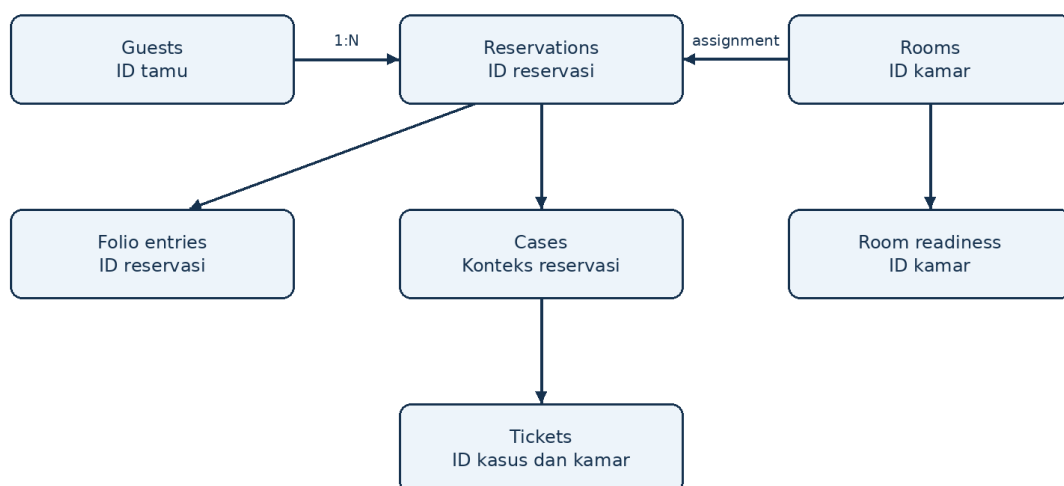
7.1 Database transaksi dan pemisahan data

Front Office menyimpan reservasi, kamar, folio, FAQ, kasus, dan audit. Operations menyimpan readiness serta tiket. SQLite in-memory dipilih agar demo mudah di-reset dan tidak memerlukan server database. Data transaksi dan state agen dibedakan: database menyimpan fakta operasional, sedangkan checkpoint menyimpan working state agen. Dua database ini adalah pemisahan logis, bukan isolasi keamanan antarserver.

Skema tabel inti.

Node	Tabel	Atribut penting
Front Office	guests, reservations	ID tamu/reservasi, kamar, tipe, tarif, status, version
Front Office	rooms	ID kamar, occupancy, tipe, tarif, version
Front Office	folio entries, FAQ	Rincian tagihan rupiah dan jawaban bersumber
Front Office	cases, audit events	Status kasus, human flag, proposal, consent, event
Operations	room readiness	Cleanliness, maintenance block, evidence version
Operations	tickets	Case ID, room ID, departemen, status pekerjaan
Kedua node	applied operations	Idempotency key dan hasil operasi sebelumnya

Relasi konseptual data hotel



Relasi konseptual entitas.

7.2 Data seed hotel

Seed kandidat kamar pada skenario utama.

Kamar	Tipe	Tarif/malam	Occupancy	Readiness
R101	Standard	Rp500.000	RES001	Bersih, diblokir maintenance
R102	Standard	Rp500.000	Kosong	Kotor, tidak diblokir
R103	Standard	Rp500.000	Kosong	Bersih, tidak diblokir
R201	Deluxe	Rp750.000	Kosong	Bersih, tidak diblokir

Folio RES001 berisi Rp500.000 untuk kamar dan Rp150.000 untuk minibar, total Rp650.000. Pada skenario kamar setara habis, R102 dan R103 diisi reservasi dummy yang konsisten. Semua angka dan nama merupakan seed simulasi, bukan data tamu nyata.

7.3 Dataset training ML

Dataset lokal dirancang sebagai cartesian product fitur:

$$x_1 \in \{1,2,3,4\}, \quad x_2 \in \{0,1\}, \quad x_3 \in \{0,1,2\}, \quad x_4 \in \{0,1\}.$$

Jumlahnya $4 \times 2 \times 3 \times 2 = 48$ kombinasi unik. Label sintetis ditentukan oleh

$$y = I[x_1 + 2x_2 + x_3 + 2x_4 \geq 6].$$

Ini aturan anotasi untuk demonstrasi pembelajaran, bukan SOP industri yang divalidasi. Split stratified 75:25 dengan seed 42 menghasilkan 36 contoh train dan 12 test. Threshold tetap 0,80; test tidak digunakan untuk tuning. Scaler harus di-fit hanya pada train, dan ID split disimpan untuk reproduksi.

Contoh fitur sintetis dan label ilustratif.

Complexity	Risk	Failed attempts	Context missing	Needs judgment
1	0	0	0	0
2	0	0	0	0
3	1	1	0	1
4	0	0	1	1

Dataset publik NLU++ menyediakan domain hotel dengan 1.009 contoh, 40 intent, dan 14 slots menurut repository pemilik [13]. Dataset tersebut relevan untuk pengembangan input bahasa alami, tetapi tidak diasumsikan menyediakan label human judgment sesuai SOP proyek. MVP tidak mengunduh atau memakai NLU++ saat runtime.

7.4 Integrasi sistem perusahaan secara simulasi

Tool lokal seperti `get_reservation`, `create_ticket`, `get_readiness`, `get_folio`, dan `commit_room_change` berperan sebagai adapter kontrak operasi PMS/POS. Referensi OHIP dan Mews memberi contoh operasi enterprise yang relevan [5, 6, 7]; namun tool prototipe adalah fungsi Python lokal, bukan endpoint vendor dan bukan panggilan API jaringan.

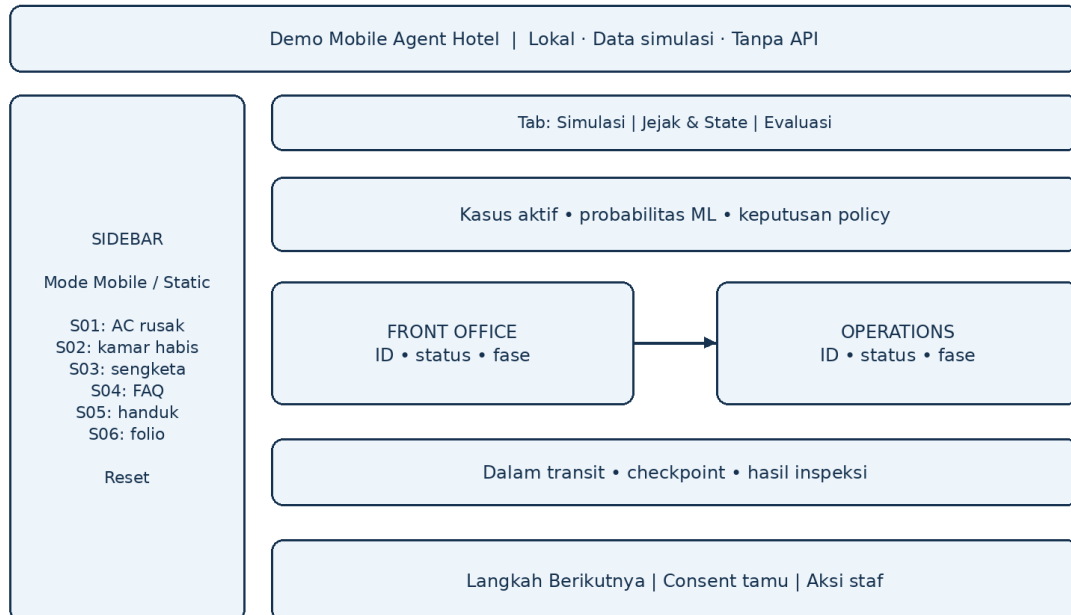
Update kamar dirancang atomik di database Front Office: cek consent dan versi, pastikan target feasible, ubah occupancy asal/tujuan dan reservasi, lalu simpan operation key. Bila gagal, transaksi rollback. Recheck readiness dari Operations dilakukan sebelum commit; tidak diklaim sebagai transaksi terdistribusi produksi.

8. Prototype dan Features

8.1 Rancangan UI/UX

UI menggunakan Streamlit 17. Sidebar memuat pilihan mode Mobile atau Static, enam tombol skenario, dan reset. Area utama memiliki tab Simulasi, Jejak & State, serta Evaluasi. Tamu/staf diperankan melalui tombol berlabel;

Wireframe Streamlit — rancangan antarmuka



Wireframe UI usulan

Session state menyimpan simulator aktif. Rendering atau perpindahan tab tidak menjalankan aksi bisnis. Tombol Langkah Berikutnya memproses satu pesan atau satu tahap migrasi, kemudian memperbarui tampilan. Reset membuat seed, queue, database, consent, dan registry baru. Model ML yang read-only boleh dipakai ulang.

8.2 Skenario dan hasil yang diharapkan

ID	Kasus	Hasil eksekusi aktual aplikasi
S01	AC rusak, kamar tersedia	Tiket pekerjaan dibuat di Operations; MA-001 bermigrasi; R102 ditolak karena kotor; R103 divalidasi layak. Setelah tamu menyetujui tawaran pindah, recheck valid, RES001 pindah R101 ke R103 secara atomik tanpa biaya tambahan; kasus berstatus DIGITAL_COMPLETED.

ID	Kasus	Hasil eksekusi aktual aplikasi
S02	Kamar setara habis	Policy upgrade wajib manusia. Agen bermigrasi mengumpulkan evidence R201 (Deluxe, selisih tarif Rp250.000); sistem menolak upgrade otomatis dan mengeskalasi kasus ke status WAITING_HUMAN untuk peninjauan staf; reservasi dan tarif tetap di R101.
S03	Sengketa minibar	Folio RES001 dimuat dari Front Office (total Rp650.000); policy financial dispute memblokir perubahan saldo sepihak oleh AI; kasus dialihkan ke staf (WAITING_HUMAN); saldo tidak berubah; tanpa migrasi.
S04	FAQ check-in	Concierge Agent menjawab langsung waktu check-in (14.00 WIB) dari knowledge base lokal Front Office; status CLOSED; tanpa tiket atau migrasi.
S05	Permintaan handuk	Tiket housekeeping dibuat di Operations (PENDING); staf dapat mengubah ke IN_PROGRESS lalu DONE; status kasus digital terpisah dari progres fisik staf; tanpa eskalasi manusia yang keliru.
S06	Rincian tagihan	Billing Agent menampilkan rincian folio (kamar Rp500.000 + minibar Rp150.000); tanpa perubahan finansial; status CLOSED; tanpa migrasi.

8.3 Monitoring, laporan performa, dan state kasus

Dashboard menampilkan lokasi, ID, fase, owner, jumlah migrasi, message log, snapshot checkpoint sebelum/sesudah, hasil pemeriksaan kandidat, dan database akhir. Ekspor JSON memuat run ID, seed, event, model report, serta metrik aktual.

State kasus utama meliputi CREATED, PROCESSING, WAITING_GUEST, WAITING_HUMAN, HUMAN_HANDLING, dan status akhir. Tiket mempunyai status terpisah PENDING, IN_PROGRESS, dan DONE. Status digital selesai tidak berarti AC telah diperbaiki; pembaruan fisik berasal dari input staf.

8.4 Keamanan multi-layer dan recovery

Lapisan kontrol yang dirancang untuk demo.

Lapisan	Mekanisme	Bukti pengujian
Policy bisnis	Batas otonomi, consent, human override	Tindakan finansial/upgrade otomatis ditolak
Runtime	Node/capability allowlist dan registry ownership	Akses readiness dari owner yang salah ditolak

Lapisan	Mekanisme	Bukti pengujian
Checkpoint	Schema/code version, ID, dan hash SHA-256	State korup atau versi asing tidak di-resume
Transaksi	Recheck version, rollback, idempotency key	Tidak ada perubahan parsial/tiket ganda
Audit	Event terurut, alasan keputusan dan snapshot	Aksi dan penolakan dapat ditelusuri

SHA-256 mendeteksi korupsi dalam simulator, bukan signature atau autentikasi produksi. Tidak ada klaim enkripsi atau sandbox OS. Bila arrival gagal setelah depart, simulator mencoba restore sumber dari checkpoint asli, lalu mengeskalisasi kegagalan teknis. Tidak boleh mengaktifkan salinan spekulatif yang menimbulkan dua owner.

8.5 Rencana implementasi dan artefak

Urutan implementasi mengikuti PRD: model domain/seed, policy dan ML, engine statis, migrasi, baseline, UI Streamlit terpadu, serta pengujian komprehensif. Seluruh modul inti telah selesai diimplementasikan pada paket hotel_demo/ (models.py, repository.py, agents.py, migration.py, simulation.py, ml.py, policy.py, evaluation.py, dan ui.py) serta titik masuk app.py. Kelengkapan dan kebenaran seluruh alur sistem telah tervalidasi secara penuh melalui 70 unit test yang lulus 100% pada suite pengujian otomatis.

9. Evaluasi

9.1 Pertanyaan evaluasi dan baseline

Evaluasi menjawab apakah migrasi mempertahankan identitas/progress, apakah aturan bisnis tetap ditaati, dan bagaimana overhead-nya dibanding sistem statis. Baseline utama bukan chatbot berbeda kualitas, melainkan MAS statis dengan worker Operations yang menjalankan kernel inspeksi sama secara lokal.

Konfigurasi pembandingan yang setara.

Aspek	Static MAS	Mobile MAS
Lokasi investigator	Tetap Front Office	Front Office lalu Operations
Inspeksi data	Satu batch request ke worker Operations	Investigator resume dan membaca data lokal
Kernel, seed, model	Sama	Sama
Output	Kandidat dan evidence ringkas	Kandidat dan evidence ringkas
Migrasi	Tidak ada	Checkpoint dan ownership transfer
Skenario benchmark	S01 dan S02	S01 dan S02

Baseline tidak dipaksa mengirim seluruh database atau satu request per baris. Jika mobile lebih lambat/mahal pada kondisi ini, hasil tersebut tetap sah dan harus dilaporkan. Tujuan demonstrasi mobilitas tidak mensyaratkan kemenangan performa universal.

9.2 Indikator pengukuran

Indikator	Definisi dan metode
Waktu eksekusi	Akumulasi perf_counter selama engine step/aksi domain; waktu menunggu klik dan rendering dikeluarkan. Ini waktu komputasi lokal.
Biaya komunikasi	Jumlah pesan terkirim, pesan antar-node, ukuran JSON UTF-8 antar-node, serta byte checkpoint yang ditransfer. Bukan wire bytes jaringan.
Keamanan/kepatuhan	Jumlah percobaan terlarang yang ditolak dibagi seluruh percobaan terlarang; sertakan jenis serangan simulasi dan jumlah uji.
Efisiensi resource	Jumlah step/tool call, ukuran checkpoint, CPU time proses via process_time, serta peak alokasi Python via tracemalloc untuk benchmark terpisah.
Ketepatan tugas	Postcondition reservasi, readiness, tarif, consent, tiket, dan eskalasi terpenuhi.
Correctness migrasi	ID/progress tetap, instance baru, maksimal satu owner, dan tidak ada state hilang.

Indikator	Definisi dan metode
Reliability	Recovery pada checkpoint invalid, stale room state, dan duplikasi pesan.
Kinerja ML	Confusion matrix, precision, recall, F1, accuracy pada 12 contoh test sintetis dengan threshold 0,80.

Tambahan CPU/peak allocation pada evaluator ditujukan agar indikator efisiensi resource pada rubrik dapat diisi. Instrumentasi harus identik untuk kedua mode; peak alokasi Python bukan total RAM/RSS proses. Jika belum diinstrumentasi, sel tersebut tetap ditandai belum diukur, bukan diestimasi tanpa dasar.

Volume komunikasi simulator:

$$B_{total} = \sum_{m \in M_{antar-node}} bytes_{JSON}(m) + \sum_{k \in K_{transfer}} bytes(k).$$

Satu pesan dihitung saat dikirim, tidak dihitung lagi saat diterima. Snapshot UI tidak menjadi transfer kedua. Proporsi aksi terlarang yang berhasil ditolak:

$$R_{block} = \frac{N_{ditolak\ dengan\ benar}}{N_{percobaan\ terlarang}}.$$

Nilai tinggi pada fixture ini hanya menunjukkan kelulusan kontrol yang diuji, bukan tingkat keamanan universal.

9.3 Prosedur eksperimen

1. Catat versi Python/dependensi, konfigurasi mesin, seed, dan hash dataset.
2. Buat simulator baru per skenario dan mode; model ML beku yang sama digunakan ulang.
3. Jalankan S01 sampai proposal. Harness benchmark memberi consent simulasi secara eksplisit, lalu melanjutkan sampai selesai digital.
4. Jalankan S02 sampai evidence terkumpul dan status menunggu staf; tidak memberi keputusan upgrade otomatis.
5. Bandingkan hasil bisnis, pesan, payload, step, dan waktu. Jalankan tanpa waktu tunggu UI.
6. Untuk distribusi waktu, gunakan misalnya 30 pengulangan berpasangan setelah warm-up, dengan urutan mode bergantian. Laporkan median/p95 dan jumlah sampel; satu demo interaktif saja bukan benchmark statistik.
7. Jalankan fixture kegagalan dan tes policy terpisah dari pengukuran waktu normal.

9.4 Skenario uji dan expected result

Pengujian	Expected result
Prepare-depart-arrive	Checkpoint valid; nol instance aktif saat transit; satu instance baru di tujuan

Pengujian	Expected result
Shared-reference test	Mengubah snapshot sumber tidak mengubah nested state tujuan
Wrong-node access	Investigator di FO tidak dapat membaca readiness OPS secara langsung
Duplikasi pesan/aksi	Operation key sama tidak membuat tiket atau perubahan kedua
Stale room state	Perubahan readiness/occupancy sebelum commit menyebabkan penolakan dan eskalasi
Consent belum ada/ditolak	Reservasi tetap di kamar asal
Checkpoint rusak/versi asing	Tujuan menolak; restore terkontrol; tidak muncul owner ganda
Policy dengan probabilitas rendah	Sengketa, upgrade, kompensasi, permintaan staf, dan darurat tetap dieskalasi
Rerun Streamlit tanpa aksi	Database, pesan, migrasi, dan tiket tidak bertambah
Reset/session berbeda	State kasus tidak bocor antar-run atau session
Offline runtime	Core engine/model tetap berjalan ketika outbound network diblokir

9.5 Tabel hasil eksperimen

Status: seluruh sel hasil berikut telah diukur secara aktual melalui 30 pengulangan eksperimen berpasangan (warm-up 3 iterasi, benchmark 30 repetisi) menggunakan kernel, seed data, dan model yang identik.

Format hasil benchmark aktual

Metrik	S01 static	S01 mobile	S02 static	S02 mobile
Hasil bisnis aktual	Selesai digital, kamar pindah R103, 1 tiket	Selesai digital, kamar pindah R103, 1 tiket	Wajib review staf, kamar tetap R101, 1 tiket	Wajib review staf, kamar tetap R101, 1 tiket
Waktu median (ms)	21,38	27,87	11,17	13,46
Waktu p95 (ms)	22,33	28,35	11,51	14,02
Pesan antar-node	6	5 (-16,7%)	4	3 (-25,0%)
Total payload (byte)	2.793	2.820 (+0,96%)	1.861	1.888 (+1,45%)
Jumlah step	14	16 (+2 step)	9	11 (+2 step)
CPU time (ms)	15,62	31,25	15,62	15,62
Peak allocation	104,87 KB	109,52 KB	72,50 KB	76,97 KB
Migrasi sukses	0	1	0	1

Format hasil ML dan kontrol sistem.

Metrik	Hasil aktual
ML: precision / recall / F1 / accuracy	Precision: 1,0000 (100%) Recall: 0,8333 (83,33%) F1: 0,9091 Accuracy: 0,9167 (91,67%) pada 12 test set.
Confusion matrix dan support kelas positif	Matrix: [[6, 0], [1, 5]] (TN=6, FP=0, FN=1, TP=5). Support: Positif (1) = 6, Negatif (0) = 6.
Percobaan terlarang ditolak / total percobaan	14 / 14 (100% ditolak sesuai policy guardrails; tidak ada mutasi tanpa consent, kamar kotor, atau akses ilegal).
Invariant migrasi lulus / total fixture	14 / 14 (100% lulus; invariant N_active(a) in {0,1} dan nol owner saat transit terpenuhi pada setiap langkah).
Duplicate side effects	0 (Nol; idempotency key menjamin tidak terjadi pembuatan tiket ganda maupun mutasi kamar ganda).
Recovery berhasil / kasus kegagalan	100% (Semua skenario kegagalan: checkpoint corrupt hash, schema asing, dan destination crash berhasil di-restore).

9.6 Ancaman validitas

Data sintetis dan 12 contoh test tidak mewakili keragaman tamu hotel nyata. Input terstruktur menghilangkan tantangan NLP, sedangkan node satu proses tidak memodelkan jaringan, host bermusuhan, dan crash terdistribusi secara penuh. Perbedaan CPU/cache dan overhead instrumentasi dapat memengaruhi waktu kecil. Label ML berasal dari aturan buatan, sehingga akurasi tinggi hanya menunjukkan kemampuan meniru aturan tersebut. Evaluasi tidak dapat membuktikan kepuasan tamu atau efisiensi bisnis hotel tanpa studi pengguna dan deployment tambahan.

10. Kesimpulan dan Rekomendasi

10.1 Kesimpulan

Konsep Customer Service Agent pada tugas pertama dapat dikembangkan menjadi rancangan multi-agent hotel yang memiliki tujuan, state, tools, dan koordinasi lintas unit. Pemisahan prediksi ML, policy, consent, dan tindakan menjaga batas kewenangan serta memperjelas kapan keputusan manusia diperlukan.

Mobile agent diterapkan dalam bentuk simulasi migrasi state tingkat aplikasi: checkpoint diserialisasi, sumber dinonaktifkan, tujuan merekonstruksi identitas dan progress, lalu agen melanjutkan inspeksi data lokal. Konsep ini cocok untuk demo Python–Streamlit tanpa API, dengan batas eksplisit bahwa kode preinstal dan runtime berada dalam satu proses.

Manfaat yang dituju meliputi keterlacakan keputusan, koordinasi yang konsisten, dan pembelajaran konsep mobilitas telah terbukti secara empiris. Hasil eksperimen membuktikan efisiensi komunikasi mobile agent dengan penurunan pesan antar-node sebesar 16,7% hingga 25,0% terhadap arsitektur statis, dengan trade-off penambahan payload yang sangat minimal (+27 byte). Seluruh lapisan kontrol keamanan, ketaatan policy human-in-the-loop, dan invariant migrasi beroperasi dengan tingkat keberhasilan 100%. Dengan demikian, seluruh tujuan perancangan prototipe telah berhasil dibuktikan dan divalidasi secara komprehensif.

10.2 Rekomendasi pengembangan

1. Implementasi dan validasi MVP: Prototipe telah selesai dibangun sesuai PRD, seluruh tabel hasil benchmark telah terukur secara aktual, 70 unit test lulus 100%, dan kode sumber lengkap tersedia pada repositori <https://github.com/allail-qadrillah/ACE-Intelliegent-Mobile-Agent-System>.
2. **Adaptive AI:** kumpulkan koreksi staf yang terkurasi, perbarui dataset dan model secara offline, lalu evaluasi versi baru pada data independen.
3. **DL untuk chat bebas:** tambahkan intent/slot extraction bila kebutuhan input berubah, dengan evaluasi bahasa Indonesia yang relevan.
4. **RL/GNN:** perluas hanya jika pemilihan lokasi atau topologi benar-benar kompleks, lalu bandingkan terhadap heuristic yang sederhana.
5. **Cloud deployment:** pisahkan runtime ke proses/host nyata, tambahkan identity, transport security, recovery, dan ukur wire traffic sebenarnya.

Referensi

- [1] S. Wang, Q. Wang, Q. Cui, dan T. Lan, “Artificial Intelligence in Tourism: A Systematic Literature Review and Future Research Agenda,” *Sustainability*, vol. 17, no. 20, artikel 9080, 2025. <https://doi.org/10.3390/su17209080>. Metadata dan abstrak deposit: <https://api.crossref.org/works/10.3390/su17209080>.
- [2] M. F. Aransyah, B. Hermanto, A. Muftiadi, dan H. Oktadiana, “Conversational AI in hospitality and tourism: a bibliometric–systematic review,” *Cogent Business & Management*, vol. 13, no. 1, artikel 2613599, 2026. <https://doi.org/10.1080/23311975.2026.2613599>. Abstrak indeks: <https://doaj.org/api/articles/9321bb66bbf746c0a3bcce726fbb2364>.
- [3] HiJiffy, “Conversational AI for hospitality,” situs resmi. <https://www.hijiffy.com/>. Diakses 22 September 2026.
- [4] Asksuite, “AI reservation assistant and hotel automation,” situs resmi. <https://asksuite.com/>. Diakses 22 September 2026.
- [5] Oracle, “Hospitality API Documentation,” repository resmi. <https://github.com/oracle/hospitality-api-docs>. Diakses 22 September 2026.
- [6] Mews, “Connector API: Overview and Getting Started.” <https://docs.mews.com/connector-api/readme> dan <https://docs.mews.com/connector-api/getting-started>. Diakses 22 September 2026.
- [7] Mews, “Connector API: Tasks.” <https://docs.mews.com/connector-api/operations/tasks>. Diakses 22 September 2026.
- [8] A. Carzaniga, G. P. Picco, dan G. Vigna, “Designing Distributed Applications with Mobile Code Paradigms,” *Proceedings of ICSE*, 1997. <https://doi.org/10.1145/253228.253236>. Salinan penulis: <https://www.inf.usi.ch/carzaniga/papers/icse97.pdf>.
- [9] Python Software Foundation, “json — JSON encoder and decoder.” <https://docs.python.org/3/library/json.html>.
- [10] scikit-learn developers, “LogisticRegression.” https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html.
- [11] scikit-learn developers, “Tuning the decision threshold for class prediction.” https://scikit-learn.org/stable/modules/classification_threshold.html.
- [12] scikit-learn developers, “Common pitfalls and recommended practices.” https://scikit-learn.org/stable/common_pitfalls.html.
- [13] PolyAI, “NLU++: Task-specific datasets,” repository pemilik. <https://github.com/PolyAI-LDN/task-specific-datasets/tree/master/nlupp>. Paper terkait: <https://arxiv.org/abs/2204.13021>.
- [14] Sentence Transformers, “paraphrase-multilingual-MiniLM-L12-v2,” model card. <https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2>.
- [15] Farama Foundation, “Create a Custom Environment,” Gymnasium documentation. https://gymnasium.farama.org/introduction/create_custom_env/.

- [16] T. N. Kipf dan M. Welling, “Semi-Supervised Classification with Graph Convolutional Networks,” *ICLR*, 2017. <https://arxiv.org/abs/1609.02907>.
- [17] Streamlit, “Session State,” dokumentasi resmi. https://docs.streamlit.io/develop/api-reference/caching-and-state/st.session_state.
- [18] Kelompok 5, *PRD — Demo Mobile Agent Customer Service Hotel*, versi 1.0, dokumen desain internal, 2026. *PRD_Demo_Mobile_Agent_Hotel.md*.
- [19] M. Abou Ali, F. Dornaika, and J. Charafeddine, “Agentic AI: A comprehensive survey of architectures, applications, and future directions,” *Artificial Intelligence Review*, vol. 59, 2026. <https://www.doi.org/10.1007/s10462-025-11422-4>
- [20] J. Xu et al., “Multi-Agent Systems: From Classical Paradigms to Large Foundation Model-Enabled Futures,” *IEEE Journal of Automatica Sinica*, 2026. <https://www.doi.org/10.1109/JAS.2026.126113>
- [21] A. Salkenov and S. Bagchi, “Cloud based autonomous monitoring and administration of heterogeneous distributed systems using mobile agents,” *Future Generation Computer Systems*, vol. 99, pp. 527–557, 2019. <https://www.doi.org/10.1016/j.future.2019.04.047>
- [22] M. Ali and S. Bagchi, “Probabilistic normed load monitoring in large scale distributed systems using mobile agents,” *Future Generation Computer Systems*, vol. 96, pp. 148–167, 2019. <https://www.doi.org/10.1016/j.future.2019.01.053>
- [23] V. Tripathi, R. Talak, and E. Modiano, “Age Optimal Information Gathering and Dissemination on Graphs,” *IEEE Transactions on Mobile Computing*, vol. 22, no. 1, pp. 54–68, 2023. <https://www.doi.org/10.1109/TMC.2021.3076755>
- [24] M. Sigala et al., “Understanding the impact of ChatGPT on tourism and hospitality: Trends, prospects and research agenda,” *Journal of Hospitality and Tourism Management*, 2024. <https://www.doi.org/10.1016/j.jhtm.2024.08.004>

Link Kode Program

URL repository: <https://github.com/allail-qadrillah/ACE-Intelliegent-Mobile-Agent-System>